

Design Tokens: Root Variables Se Shuruaat

Ab tak humne responsive *behavior* seekha — ab baat karte hain reusability ki. Senior frontend developers jab CSS likhte hain, to unka pehla step hota hai: **hardcoded values (colors, spacing, fonts) ko kabhi directly rules mein na likhna**. Iski jagah, sab values ko ek jagah — `:root` — mein **custom properties (CSS variables)** ke tor par define karte hain.

BEGINNER APPROACH (GALAT)

```
.btn-primary {
  background: #238636;
}
.link {
  color: #238636; /* same green, dobara likha */
}
```

Problem: agar brand color kal change ho jaye, to har jagah dhoond dhoond kar replace karna parega — jaisa wrapper ke multiple fixed widths wala anti-pattern tha, wahi masla colors ke sath.

SENIOR APPROACH — DESIGN TOKENS

```
:root {
  /* Colors */
  --color-primary: #238636;
  --color-primary-hover: #2ea043;
  --color-danger: #da3633;
  --color-text: #0d1117;
  --color-muted: #57606a;
  --color-surface: #ffffff;
  --color-border: #d0d7de;

  /* Spacing scale */
  --space-1: 4px;
  --space-2: 8px;
  --space-3: 16px;
  --space-4: 24px;
  --space-5: 40px;

  /* Radius / Shadow / Font */
  --radius-sm: 6px;
  --radius-md: 10px;
  --shadow-sm: 0 1px 2px rgba(0,0,0,.08);
  --font-sans: 'Inter', system-ui, sans-serif;
}

.btn-primary { background: var(--color-primary); }
.link { color: var(--color-primary); }
```

--color-primary	--color-danger	--color-muted	--color-border

GOLDEN RULE

Agar koi value (color, spacing, radius) do ya do se zyada jagah use ho rahi hai, to wo ek variable ban'ni chahiye. Yeh "single source of truth" wala principle hai — same jaisa max-width wrapper ka tha: **ek jagah define karo, har jagah reference karo**.

Kyun sirf "hardcode kar dete hain" kaafi nahi?

- **Theming:** Dark mode add karna ho to sirf `:root` ke values `[data-theme="dark"]` ke andar override karne hote hain — poori CSS dobara likhne ki zaroorat nahi.
- **Consistency:** Designer ne jo spacing scale di hai (4/8/16/24/40) usay follow karna easy ho jata hai jab wo variables mein already available ho.
- **Maintainability:** Rebrand ho jaye to ek line change karke poori site update ho jaati hai.

BEM Methodology — Predictable Class Names

Reusable components banane ke liye sirf variables kaafi nahi — class names bhi consistent aur predictable hone chahiye. Sabse widely-used convention **BEM (Block, Element, Modifier)** hai.

BEM SYNTAX

```
.block{ }                /* .card */
.block__element{ }       /* .card__title */
.block--modifier{ }      /* .card--featured */
.block__element--modifier{ } /* .card__title--large */
```

Part	Matlab	Example
Block	Standalone, reusable component	.card, .btn, .modal
Element	Block ka child, jo block ke bahar meaningless hai	.card__body, .navbar__link
Modifier	Block/element ka variation ya state	.btn--outline, .alert--error

BEM KYUN BEHTAR HAI

- › Nested selectors (.card .title) avoid ho jate hain — jo specificity wars aur unpredictable overrides create karte hain.
- › Class name se hi pata chal jata hai component kahan belong karta hai — .navbar__link dekh kar samajh aata hai yeh navbar ka element hai.
- › Flat specificity — sab single classes hain, is liye ek dusre ko override karna predictable rehta hai.

Utility-First Alternative (Tailwind-style)

Ek dusra popular approach: chhoti, single-purpose utility classes jo directly HTML mein combine hoti hain (jaise flex gap-3 p-4 rounded-md). Yeh BEM se opposite philosophy hai — components CSS files mein nahi, HTML mein compose hote hain.

	BEM (Component classes)	Utility-First
CSS file size	Grow karti hai jitne components utne hi hain	Ek dafa stable ho jati hai
HTML readability	Saaf — class="card"	Verbose — kaafi classes ek sath
Reuse ka level	Component-level	Property-level

PRACTICAL TAKEAWAY

Ye notes BEM + design tokens ka combination use karenge, kyunke yeh CSS ko plain, framework-free HTML/CSS ke saath samajhna aasan banata hai — jo is series ka goal hai.

Button — Base + Variants + Sizes + States

Ek achi button system mein **ek base class** hoti hai jo shared styles deti hai, aur **modifier classes** jo sirf farq (color, size) declare karti hain — poori style dobara nahi likhni parti.



```
<!-- HTML: sirf classes combine karke variant banti hai -->
<button class="btn btn--primary">Save</button>
<button class="btn btn--outline btn--sm">Cancel</button>
<button class="btn btn--danger" disabled>Delete</button>
```

```
/* Base — sab buttons ki shared style */
.btn {
  display: inline-flex;
  align-items: center;
  gap: var(--space-2);
  padding: 10px 20px;
  border-radius: var(--radius-sm);
  border: 1px solid transparent;
  font-weight: 600;
  font-family: var(--font-sans);
  cursor: pointer;
  transition: background .15s ease;
}

/* Modifiers — sirf jo farq hai wahi likhna */
.btn--primary { background:var(--color-primary); color:#fff; }
.btn--primary:hover{ background:var(--color-primary-hover); }
.btn--outline { background:transparent; color:var(--color-primary); border-color:var(--color-primary); }
.btn--danger { background:var(--color-danger); color:#fff; }

/* Sizes */
.btn--sm { padding: 6px 14px; font-size: .85em; }
.btn--lg { padding: 14px 28px; font-size: 1.1em; }

/* State — attribute selector, JS class ki zaroorat nahi */
.btn:disabled { opacity: .5; cursor: not-allowed; }
```

REUSABILITY KA PRINCIPLE

Har naya button variant banane ke liye **sirf ek modifier class** add hoti hai — base structure kabhi copy-paste nahi hota. Yehi "DRY" (Don't Repeat Yourself) ka core idea hai jo poori component-based CSS ki bunyaad hai.

Card — Header / Body / Footer Structure

Card ek container component hai jo apne andar related content group karta hai (image, title, description, action). Reusability ke liye card ko **fixed inner parts** mein todna hota hai taake koi bhi content wahan drop kiya ja sake.



Card Title

Chhota description text jo card ke andar fit hota hai.

Bina Image Wala Card

Image optional hai — structure flexible rehta hai.

```
<div class="card">
  
  <div class="card__body">
    <h3 class="card__title">Title</h3>
    <p class="card__text">Description...</p>
  </div>
  <div class="card__footer">
    <button class="btn btn--primary btn--sm">View</button>
  </div>
</div>
```

```
.card {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  overflow: hidden; /* image ke corners bhi round rahein */
  box-shadow: var(--shadow-sm);
}
.card__media { display: block; width: 100%; aspect-ratio: 16/9; object-fit: cover; }
.card__body { padding: var(--space-3); }
.card__title { margin-bottom: var(--space-1); }
.card__footer { padding: var(--space-3); border-top: 1px solid var(--color-border); }
```

NOTE KIYA GAYA CONNECTION

.card__media par woh same display: block aur object-fit pattern hai jo Section 27 (Responsive Images) mein cover hua tha — reusable components purani responsive fundamentals ko hi structured tareeqe se use karte hain, kuch naya "magic" nahi hai.

Alert — Status Variants Ka Pattern

Alerts (success/warning/error/info) sabse simple example hain "base + modifier" ka — sirf color aur icon farq hota hai, structure same rehta hai.

✓ Successfully saved changes.

⚠ Yeh action undo nahi ho sakega.

✗ Save nahi hua — network error.

i Naya update available hai.

```
.alert {  
  padding: var(--space-2) var(--space-3);  
  border-radius: var(--radius-sm);  
  border-left: 4px solid currentColor;  
  display: flex;  
  gap: var(--space-2);  
}  
  
/* currentColor trick: border khud text-color follow kar leta hai */  
.alert--success { background:#f0fdf4; color:#166534; }  
.alert--warning { background:#fffbeb; color:#92400e; }  
.alert--error { background:#fef2f2; color:#991b1b; }  
.alert--info { background:#e6f2ff; color:#1e40af; }
```

ACCESSIBILITY REMINDER

Alert HTML mein role="alert" (urgent messages ke liye) ya role="status" (non-urgent updates) add karna chahiye, taake screen readers ise announce kar sakein — sirf visual color kaafi nahi hota, kyunke color-blind users ke liye success aur error mushkil se distinguish ho sakte hain.

Modal — Overlay + Dialog + State Toggle

Modal do parts se milta hai: **overlay** (background dim karne wali layer) aur **dialog** (actual content box). Reusable banane ke liye ismein CSS ke sath thoda JS bhi chahiye hota hai — sirf ek class toggle karke show/hide hota hai.

```
<div class="modal-overlay" id="myModal" hidden>
  <div class="modal" role="dialog" aria-modal="true" aria-labelledby="modalTitle">
    <header class="modal__header">
      <h3 id="modalTitle">Confirm</h3>
      <button class="modal__close" aria-label="Close">&times;</button>
    </header>
    <div class="modal__body">Kya aap sure hain?</div>
    <footer class="modal__footer">
      <button class="btn btn--secondary">Cancel</button>
      <button class="btn btn--danger">Delete</button>
    </footer>
  </div>
</div>
```

```
.modal-overlay {
  position: fixed; inset: 0;
  background: rgba(0,0,0,.5);
  display: grid; place-items: center;
}
.modal {
  background: var(--color-surface);
  border-radius: var(--radius-md);
  width: min(90vw, 420px); /* Section 22 ka clamp-jaisa min() pattern */
}
```

```
// JS: sirf hidden attribute toggle karna
document.querySelector('.modal__close').addEventListener('click', () => {
  document.getElementById('myModal').hidden = true;
});
```

GOLDEN RULE

hidden HTML attribute use karna `display:none` class toggle karne se zyada semantic hai, aur `[hidden]{display:none}` already browser default hai. Reusable component ka structure hamesha same rehta hai — sirf state (hidden/visible) badalti hai.

ACCESSIBILITY (ZAROORI)

Real production modal mein focus trap chahiye hota (Tab key modal ke andar hi cycle kare), aur Escape key se close hona chahiye. Yeh notes sirf CSS/HTML structure cover karte hain — production-grade accessibility ke liye ek dedicated library (jaise native `<dialog>` element) recommend ki jaati hai.

Navbar — Flexbox + Mobile Toggle

Navbar wahi flex-wrap aur "adding complexity" min-width media query pattern use karta hai jo Section 06-07 mein cover hua tha — bas ab ek reusable component ke tor par structured hai.

```
<nav class="navbar">
  <a class="navbar__brand" href="/">Logo</a>
  <input type="checkbox" id="navToggle" class="navbar__toggle-input">
  <label for="navToggle" class="navbar__toggle" aria-label="Menu">≡</label>
  <ul class="navbar__links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
  </ul>
</nav>
```

```
.navbar {
  display: flex; align-items: center;
  justify-content: space-between;
  padding: var(--space-3);
}
.navbar__toggle-input, .navbar__toggle { display: none; }
.navbar__links { display: flex; gap: var(--space-4); list-style: none; }

/* Mobile: base state = collapsed menu, "adding complexity" upar ki taraf hoti hai */
@media (max-width: 640px) {
  .navbar__toggle { display: block; cursor: pointer; }
  .navbar__links {
    display: none; flex-direction: column;
    position: absolute; top: 100%; inset-inline: 0;
  }
  .navbar__toggle-input:checked ~ .navbar__links { display: flex; }
}
```

"CHECKBOX HACK" — TRADE-OFF

Yeh JavaScript-free tareeqa hai (hidden checkbox + label + :checked ~ sibling selector), demos aur simple sites ke liye theek hai. Production apps mein screen-reader announcement aur keyboard behavior ke liye **proper JS toggle (aria-expanded ke sath)** zyada reliable hota hai — is liye yeh sirf CSS-only pattern samajhne ke liye demonstrate kiya gaya.

Reusable CSS Likhne Ke Golden Rules

Yeh saari cheezein jo upar dekhi — variables, BEM, base+modifier — inka combined result senior-level conventions ki ek list ban'ta hai jo har professional codebase mein follow hoti hai.

1. Layered Architecture (ITCSS-style order)

Layer	Kya Hota Hai
Settings	:root variables — colors, spacing, fonts
Base	Resets, <code>*{box-sizing}</code> , default body/img rules
Components	<code>.btn</code> , <code>.card</code> , <code>.modal</code> — reusable blocks
Utilities	Chhoti helper classes (<code>.text-center</code> , <code>.mt-4</code>) — sabse last, sab ko override kar sakein

2. Specificity Ko Low Aur Flat Rakho

AVOID

```
.sidebar .card .title span { ... } /* deeply nested, fragile */
```

PREFER

```
.card__title-highlight { ... } /* flat, ek hi jagah override hota hai */
```

3. !important Se Bachein

!important ek "escape hatch" hai jo specificity system ko poora tor par tor deta hai — future mein us rule ko override karna almost namumkin ho jata hai. Iski jagah, low specificity selectors aur sahi source-order use karein.

4. Mobile-First Likhein

Base CSS mobile ke liye (bina media query), phir min-width queries mein complexity add karein — yeh Section 07 mein already establish ho chuka principle hai, aur components ke liye bhi wahi apply hota hai.

5. rem/em Scalability Ke Liye, px Precision Ke Liye

Font-sizes aur spacing jo user ke zoom/preference ke sath scale hone chahiye — rem. Borders, fine spacing jo consistent rehni chahiye — px theek hai (Section 21 se yaad rahe: layout-level spacing ke liye px valid choice hai).

6. Component Folder Structure

```
styles/
  tokens.css      /* :root variables */
  base.css        /* resets */
  components/
    button.css
    card.css
    modal.css
    navbar.css
  utilities.css
```

OVERALL TAKEAWAY

Reusability koi alag skill nahi hai — yeh wahi responsive mindset (browser ko hints do, absolutist mat bano) ka extension hai, lekin ab **component level** par apply ho raha hai: ek base likho, variations ko modifiers se control karo, aur values ko tokens mein rakho.

Practice Questions — Reusable Patterns

Q1. Design tokens kya hote hain aur inhe kahan define kiya jata hai?

Reusable values (colors, spacing, radius, fonts) jo CSS custom properties ke tor par :root mein define hoti hain, taake ek single source of truth rahe.

Q2. BEM ke teen parts kya hain aur unka syntax kya hai?

Block (.card), Element (.card__title), Modifier (.card--featured).

Q3. Button component mein "base + modifier" approach kyun better hai copy-paste se?

Shared styles ek jagah (.btn) define hoti hain, aur naya variant banane ke liye sirf ek chhoti modifier class add hoti hai — DRY principle follow hota hai.

Q4. Card component ke teen standard inner parts kaunse hain?

Media/Header, Body, Footer.

Q5. Alert mein currentColor trick kya karta hai?

Border ka color automatically text color follow kar leta hai, is liye alag se border-color declare nahi karna parta.

Q6. Modal ke liye hidden attribute use karna class-toggle se better kyun hai?

Zyada semantic hai aur [hidden]{display:none} already browser default behavior hai — extra CSS likhne ki zaroorat nahi.

Q7. Navbar ke "checkbox hack" ka trade-off kya hai?

JS-free hai lekin accessibility (screen readers, keyboard) production apps ke liye kamzor rehti hai — asli projects mein JS + aria-expanded behtar hai.

Q8. !important se kyun bachna chahiye?

Yeh specificity system ko tor deta hai, jisse future mein us rule ko override karna bohot mushkil ho jata hai.

Q9. ITCSS-style layering mein utilities layer sabse last kyun aati hai?

Taake unki low specificity ke bawajood woh baaki sab layers (settings, base, components) ko safely override kar sakein, source-order ki wajah se.

Q10. rem aur px ke istemal ka general rule kya hai reusable components mein?

Font-size/spacing jo user preference ke sath scale honi chahiye — rem. Fine, consistent layout details (borders) — px theek hai.

FILHAL YAHAN TAK

Yeh section CSS notes ka extension hai — reusable UI patterns (buttons, cards, alerts, modals, navbars) aur senior-level CSS conventions cover karta hai. Agla step: inhe apne project mein practically implement karna, ya kisi specific component (jaise dropdown, tabs, tooltip) ko is document mein add karna.